SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA0302	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	SAMUEL K	**Reg. No.:**	192424063
Section / Batch:	B. TECH (AI&DS)	**Date:**	20/10/2026

Code of Conduct

*I _____**SAMUEL K**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____SAMUEL K_____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. **Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.**

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

CODE:

```
import re

words = ["connected", "connecting", "connection"]

def morphological_analysis(word):
    if re.search(r"ied$", word):
        root = re.sub(r"ied$", "y", word)
        suffix = "-ied"
        category = "Inflectional"

    elif re.search(r"ed$", word):
        root = re.sub(r"ed$", "", word)

        if root.endswith("ct"):
            root += "e"

        suffix = "-ed"
        category = "Inflectional"

    elif re.search(r"ing$", word):
        root = re.sub(r"ing$", "", word)

        if root.endswith("ct"):
            root += "e"

        suffix = "-ing"
        category = "Inflectional"
```

```
    elif re.search(r"ion$", word):
        root = re.sub(r"ion$", "", word) + "t"
        suffix = "-ion"
        category = "Derivational"

    else:
        root = word
        suffix = "None"
        category = "Base form"

    return root, suffix, category

print(f"{'Word':<15}{'Root':<15}{'Suffix':<12}{'Type':<18}{'Normalized'}")
print("-" * 75)

for word in words:
    root, suffix, category = morphological_analysis(word)
    print(f"{word:<15}{root:<15}{suffix:<12}{category:<18}connect")
```

OUTPUT:

```
Word           Root           Suffix      Type               Normalized
---------------------------------------------------------------------------
connected      connecte       -ed         Inflectional       connect
connecting     connecte       -ing        Inflectional       connect
connection     connectt       -ion        Derivational       connect
```

2.Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

CODE:

```
import re
words = ["unhappy", "happiness", "happily"]
def morphological_analysis(word):
    prefix = "None"
    suffix = "None"
    root = word
    if re.match(r"^un", word):
        prefix = "un-"
        root = re.sub(r"^un", "", word)
        category = "Derivational"
        meaning = "Negation or opposite meaning"
    elif re.search(r"ness$", word):
        suffix = "-ness"
        root = re.sub(r"ness$", "", word)

        # happiness -> happy
        if root.endswith("i"):
            root = root[:-1] + "y"

        category = "Derivational"
        meaning = "Forms a noun expressing a state"

    elif re.search(r"ly$", word):
        suffix = "-ly"
        root = re.sub(r"ly$", "", word)

        # happily -> happy
        if root.endswith("i"):
            root = root[:-1] + "y"

        category = "Derivational"
        meaning = "Forms an adverb"

    else:
        category = "Base form"
        meaning = "Base meaning"

    return prefix, root, suffix, category, meaning

print(f"{'Word':<15}{'Prefix':<12}{'Root':<15}{'Suffix':<12}{'Type':<18}{'Meani
ng'}")
print("-" * 100)

for word in words:
    prefix, root, suffix, category, meaning = morphological_analysis(word)
    print(f"{word:<15}{prefix:<12}{root:<15}{suffix:<12}{category:<18}{meaning}
")
```

OUTPUT:

Word	Prefix	Root	Suffix	Type	Meaning
unhappy	un-	happy	None	Derivational	Negation or opposite meaning
happiness	None	happy	-ness	Derivational	Forms a noun expressing a state
happily	None	happy	-ly	Derivational	Forms an adverb

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

CODE:

```
import re
from nltk.stem import PorterStemmer

words = ["played", "player", "playing"]
stemmer = PorterStemmer()

def analyze_word(word):
    if re.search(r"ed$", word):
        affix = "-ed"
        category = "Inflectional"
    elif re.search(r"er$", word):
        affix = "-er"
        category = "Derivational"
    elif re.search(r"ing$", word):
        affix = "-ing"
        category = "Inflectional"
    else:
        affix = "None"
        category = "Base form"

    stem = stemmer.stem(word)

    return stem, affix, category

print(f"{'Word':<15}{'Stem':<15}{'Affix':<12}{'Transformation':<18}{'Normalized
'}")
```

```
print("-" * 75)

for word in words:
    stem, affix, category = analyze_word(word)
    print(f"{word:<15}{stem:<15}{affix:<12}{category:<18}{stem}")
```

OUTPUT:

```
Word           Stem           Affix       Transformation    Normalized
---------------------------------------------------------------------------
played         play           -ed         Inflectional      play
player         player         -er         Derivational      player
playing        play           -ing        Inflectional      play
```

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

CODE:

```
import re

words = ["writes", "writing", "written"]

# Finite-state transition model
transitions = {
    "START": {"writes": "REGULAR_S", "writing": "REGULAR_ING",
              "written": "IRREGULAR_PARTICIPLE"},
    "REGULAR_S": "ACCEPT",
    "REGULAR_ING": "ACCEPT",
    "IRREGULAR_PARTICIPLE": "ACCEPT"
}

def finite_state_parse(word):
    if re.search(r"s$", word):
        state_path = "START -> REGULAR_S -> ACCEPT"
        root = re.sub(r"s$", "", word)
```

```
        category = "Regular inflection"
        affix = "-s"

    elif re.search(r"ing$", word):
        state_path = "START -> REGULAR_ING -> ACCEPT"
        root = re.sub(r"ing$", "", word) + "e"
        category = "Regular inflection"
        affix = "-ing"

    elif word == "written":
        state_path = "START -> IRREGULAR_PARTICIPLE -> ACCEPT"
        root = "write"
        category = "Irregular inflection"
        affix = "Irregular form"

    else:
        state_path = "START -> UNKNOWN"
        root = word
        category = "Unknown"
        affix = "None"

    return state_path, root, affix, category

print(f"{'Word':<12}{'Root':<12}{'Affix':<18}{'Type':<22}{'State path'}")
print("-" * 100)

for word in words:
    path, root, affix, category = finite_state_parse(word)
    print(f"{word:<12}{root:<12}{affix:<18}{category:<22}{path}")
```

OUTPUT:

```
Word        Root        Affix             Type                  State path
----------------------------------------------------------------------------------------------------
writes      write       -s                Regular inflection    START -> REGULAR_S -> ACCEPT
writing     write       -ing              Regular inflection    START -> REGULAR_ING -> ACCEPT
written     write       Irregular form    Irregular inflection  START -> IRREGULAR_PARTICIPLE -> ACCEPT
```

5. Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present

the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

CODE:

```
from nltk.stem import PorterStemmer

words = ["relational", "relation", "relate"]
stemmer = PorterStemmer()

def stemming_steps(word):
    steps = [word]
    current = word

    # Demonstrate common derivational suffix processing
    if current.endswith("ational"):
        current = current[:-7] + "ate"
        steps.append(current)

    current = stemmer.stem(word)

    if current != steps[-1]:
        steps.append(current)

    return steps, current

print(f"{'Word':<15}{'Intermediate forms':<35}{'Final stem'}")
print("-" * 70)

for word in words:
    steps, stem = stemming_steps(word)
    print(f"{word:<15}{' -> '.join(steps):<35}{stem}")
```

OUTPUT:

```
Word           Intermediate forms                 Final stem
----------------------------------------------------------------------
relational     relational -> relate -> relat      relat
relation       relation -> relat                  relat
relate         relate -> relat                    relat
```

Rubrics for Calculation

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____